

MADLearn

User Manual

Detailed functional guide for learners, administrators, and operators

Document	Operational reference
Application	MADLearn - AI-powered learning management for South African business teams
Prepared	26 May 2026

Built for the MAD Prospects universe: learning, assessment, progress evidence, admin insight, and MAD Cloud task validation.

1. Executive Overview

MADLearn is an AI-powered learning management system in the MAD Prospects ecosystem. It delivers structured course content, lesson navigation, tests, progress tracking, administrative oversight, and a MAD Cloud validation task surface through a secured web application.

The current application is built around a practical bootcamp learning model: users authenticate, browse available courses, work through day-based modules and lessons, complete quizzes, and accumulate progress evidence. Administrators can monitor learner status, test results, pass rates, and individual user histories.

- * Public visitors can read the landing page, compare pricing tiers, sign in, or register.
- * Authenticated learners can view dashboards, enroll themselves into available courses by opening them, read lessons, copy code examples, take tests, and track progress.
- * Administrators can view all users, inspect each learner's progress and tests, and monitor global test performance.
- * The MAD Cloud page allows authenticated users to submit a validation task and receive a completed worker response.
- * The backend protects learning, progress, testing, admin, and MAD Cloud endpoints with JWT authentication and role checks.

2. Application Architecture

MADLearn is split into a modern Angular frontend and an ASP.NET Core backend. The frontend is a standalone Angular application using Angular Material and Tailwind-style utility classes. The backend exposes REST endpoints, stores data through Entity Framework Core, and uses SQL Server in production.

Area	What it does	Who uses it
Frontend	Angular standalone routes, Material controls, forms, dashboards, and authenticated shell navigation.	Learners, admins, public visitors
Backend API	ASP.NET Core controllers for auth, courses, modules, lessons, tests, progress, admin, health, and MAD Cloud.	Frontend and integrations
Data model	Users, courses, modules, lessons, tests, questions, answers, progress records, test results, and MAD Cloud tasks.	System operators and developers
Security	JWT login, bearer token interceptor, route guards, and Admin role authorization for admin APIs.	All roles
Deployment	Production frontend and API run on madprospects.com subdomains with SQL Server backing services.	Operators

3. Roles and Permissions

The application currently uses two practical roles: Student and Admin. A newly registered user is created as Student. Admin users receive the same learner functionality plus the administrative dashboard and API access.

- * Student: can sign in, browse courses, view modules and lessons, complete lessons, take tests, view their progress, view their test results, and submit MAD Cloud validation tasks.

- * Admin: can do everything a Student can do, plus view the Admin Dashboard, inspect users, see lesson completion and test metrics, and open individual learner detail pages.
- * Superadmin bootstrap: the backend ensures that admin@madprospects.com exists as an Admin account during startup seeding so both development and production have a known administrative entry point.

Operational note: because the bootstrap account is powerful, its password should be rotated and managed through a team password manager once deployment is stable.

4. Public Landing Page

The public home page introduces MADLearn as the MAD Prospects LMS. It positions the product as training software for South African teams and highlights branded course delivery, learner progress evidence, and MAD Cloud task execution.

- * Top navigation presents the MADLearn brand, Sign in, and Open app actions.
- * Hero content explains the product promise: structured learning management, progress tracking, assessments, and MAD Cloud validation.
- * The MAD Universe strip links to other MAD Prospects applications such as MADai, MADAuthor, MADCreate, MADHub, MADLeads, MADLove, MADPulse, MADRecruiting, and others.
- * Pricing cards describe Starter, Team, and Academy tiers for training adoption and scale.
- * Calls to action route visitors to login, registration, or the learner dashboard if already authenticated.

5. Authentication

Users authenticate with email and password. Successful login or registration returns a JWT token and a small user profile containing username, email, role, and user id. The frontend stores these in local storage and attaches the token to later API requests.

1. Open the sign-in page.
2. Enter a registered email and password.
3. Submit the form.
4. If credentials are valid, MADLearn stores the session and routes the user to the dashboard.
5. If credentials are invalid, the form displays a clear login failure message.

Registration collects username, email, and password. Username must be at least three characters, email must be valid, and password must be at least six characters. Registration creates a Student role user and logs them in immediately.

Logout removes the stored token and user profile, then returns the user to the login page.

6. Authenticated Shell

After login, the user enters a left-sidebar application shell. The shell gives consistent navigation and identity context across all protected pages.

- * Dashboard: the learner's home view with progress stats and available courses.
- * My Progress: detailed lesson progress and test result history.
- * MAD Cloud: task submission and completion display.
- * Admin Dashboard: visible only when the current user role is Admin.
- * User card: shows username and role, with a Logout button.

7. Learner Dashboard

The dashboard is the learner's main operating surface. It combines summary metrics, overall progress, available courses, and recent test results.

- * Lessons Done shows the count of completed lessons.
- * Progress shows completed lessons as a percentage of all lessons in the system.
- * Tests Taken shows the number of recorded test attempts returned for the current user.
- * Time Spent summarizes minutes tracked across lessons and displays them in minutes or hours.
- * Overall Progress visualizes the ratio of completed lessons to total lessons.
- * Available Courses shows each course with title, description, tech stack chips, duration, hours per day, module count, and a Start Learning action.
- * Recent Test Results lists up to five latest test results, including title, date, score, and pass/fail badge.

8. Courses, Modules, and Lessons

The course model is hierarchical: a course contains modules, modules contain lessons, and lessons may have an associated test. The current seeded course is Full-Stack Developer Bootcamp, a two-week program covering TypeScript, Angular, .NET 8, and SQL Server.

Area	What it does	Who uses it
Course	Title, description, tech stack, duration in days, hours per day, and module count.	Learner dashboard and course page
Module	Day number, title, description, order index, and lesson count.	Course view
Lesson	Title, markdown-like content, optional code example, type, estimated minutes, and test availability.	Lesson viewer
Test	Lesson-linked quiz with passing score, questions, answers, and scored result.	Test screen and reporting

The course view groups modules by day. Expanding a module shows lessons, completion status, lesson type, estimated minutes, and whether a quiz exists. Completed lessons are visually marked; unfinished lessons show an unchecked state.

The lesson viewer has a main content area and an action sidebar. Content is rendered from simple markdown-like text into headings, paragraphs, emphasis, lists, inline code, and code blocks. If a lesson includes a code example, a separate Code Example tab appears with a Copy action.

- * Mark Complete records the lesson as completed and sends elapsed minutes to the backend.
- * Take Quiz appears when a test is attached to the lesson.
- * The time tracker runs while the lesson page is open and sends time spent when the user leaves the lesson.
- * Completed lessons display a success badge when revisited.

9. Tests and Assessment

MADLearn supports lesson-linked multiple-choice tests. Tests are loaded by lesson id and presented one question at a time. The user selects one answer per question, moves forward and backward, jumps by question number, and submits when all questions are answered.

- * The test header shows title, passing score, and question progress.

- * Answer cards visually indicate the selected answer.
- * Navigation controls include Previous, Next, Submit Test, and per-question buttons.
- * Submission sends question and answer ids to the API.
- * The backend calculates score as correct answers divided by total questions, rounds to a whole percentage, compares it to the passing score, and stores a `TestResult`.
- * The result screen shows a pass or fail state, the score, the test title, and navigation back to the lesson or dashboard.

The seeded TypeScript tests include questions on typed variables, primitive types, readonly properties, type inference, constants, optional interface properties, class implementation of interfaces, extending interfaces, and union types.

10. Progress Tracking

Progress tracking is designed to capture both completion and learning effort. A unique progress record is maintained per user and lesson.

- * Completing a lesson creates or updates a `UserProgress` record, sets `IsCompleted` to true, stores `CompletedAt`, and adds time spent.
- * Time tracking creates or updates progress even if the lesson is not completed yet.
- * The summary endpoint counts total lessons, completed lessons, completion percentage, and total time.
- * Progress is personal: each user sees only their own lesson progress and test result history.

The My Progress page presents three top metrics: lessons completed, course progress percentage, and hours studied. It also provides two tabs: Lesson Progress and Test Results.

11. MAD Cloud Task Validation

The MAD Cloud page is an authenticated task submission surface. In the current implementation, it behaves as a local worker validation flow rather than a full distributed compute platform.

1. The user opens MAD Cloud from the sidebar.
2. The page pre-fills a validation smoke-task input.
3. The user submits the task.
4. The API creates a `MadCloudTask` record with status `Running`.
5. The local worker logic immediately writes a completed output message, marks status `Completed`, stores completion time, and returns the task.
6. The frontend displays task id, status, completed timestamp, and output text.

This is useful as an integration seam: it proves authenticated task creation, task ownership, task status, and task result rendering. It is ready to be expanded into real MAD Cloud job orchestration.

12. Admin Dashboard

The Admin Dashboard is available only to users whose JWT role is Admin. It provides high-level learner and assessment visibility.

- * Total Students displays the count of users returned by the admin users endpoint.
- * Tests Taken counts all test result records returned by the admin test-results endpoint.
- * Avg Score calculates the average of all test result scores.
- * Pass Rate calculates the percentage of results where `Passed` is true.

- * The Students tab lists username, email, role, completed lessons, total tests, average score, joined date, and a View Details action.
- * Search filters users by username or email.
- * The Test Results tab lists recent results with test title, date/time, score, and pass/fail badge.

The User Detail page opens from the Admin Dashboard. It shows the user's profile, role, joined date, completed lesson count, total tests, average score, lesson progress records, time spent, and test results.

13. Data and API Reference

The backend data model is simple and intentionally close to the learning workflow.

Area	What it does	Who uses it
Users	Identity, email, password hash, role, created date, related progress, test results, and MAD Cloud tasks.	Auth and admin
Courses	Training programs with title, description, tech stack, duration, hours per day, and modules.	Course catalog
Modules	Day-based groupings of lessons inside a course.	Course navigation
Lessons	Learning content with optional code example, type, order, estimated minutes, and optional test.	Lesson viewer
Tests	Quiz definition with passing score and questions.	Assessment
Questions/Answers	Multiple-choice question and answer options, including correctness flag stored server-side.	Assessment
UserProgress	Per-user lesson completion and accumulated minutes.	Progress tracking
TestResults	Per-user test score, pass/fail flag, and timestamp.	Reporting
MadCloudTasks	Per-user task input, type, status, output, and timestamps.	MAD Cloud

14. Security Behavior

- * Passwords are stored as BCrypt hashes, not plain text.
- * JWT tokens include user id, email, username, and role claims.
- * Tokens expire after seven days.
- * The frontend HTTP interceptor adds Authorization: Bearer tokens to API calls when logged in.
- * Angular route guards redirect unauthenticated users to login and redirect non-admin users away from admin routes.
- * Backend controllers require authorization for courses, modules, lessons, tests, progress, admin, and MAD Cloud routes.
- * Admin endpoints require the Admin role at the API level, not only in the UI.
- * CORS allows configured frontend origins for development and production.

15. Operational Notes

- * Production frontend URL: <https://madlearn.madprospects.com>.
- * Production API URL: <https://madlearnapi.madprospects.com/api>.
- * The health endpoint reports product, database connection status, and timestamp.
- * Database startup applies migrations when possible and still attempts seed/bootstrap for existing tables.
- * Seed data creates the bootcamp course, modules, lessons, tests, and ensures a superadmin account.
- * For local development, the API is expected on port 3016 and the frontend on port 4216.

16. Common User Workflows

New learner first session

1. Open the MADLearn public site and choose Create account.
2. Enter username, email, and password. The account is created as a Student.
3. Arrive at the dashboard and review the available course cards.
4. Open a course and expand the day/module structure.
5. Start the first lesson, read the content, and inspect any code example.
6. Use Mark Complete when the lesson has been studied.
7. Take the quiz if one is attached to the lesson.
8. Return to Dashboard or My Progress to confirm progress and results were stored.

Returning learner study session

1. Sign in with the existing email and password.
2. Use Dashboard to see lessons completed, percentage progress, tests taken, and time spent.
3. Open the active course and look for unchecked lessons.
4. Review completed lessons when needed, or continue with the next unfinished lesson.
5. Use My Progress to check detailed lesson status, time spent, and test history.

Administrator review session

1. Sign in as an Admin user.
2. Open Admin Dashboard from the sidebar.
3. Review top-level metrics: total users, test attempts, average score, and pass rate.
4. Search for a learner by username or email.
5. Open the learner detail page.
6. Inspect lesson progress, time spent, completion status, test count, and average score.
7. Use the Test Results tab to identify failed tests, weak areas, and recent assessment activity.

17. API Behavior by Feature

Area	What it does	Who uses it
Authentication	POST /auth/register creates a Student account. POST /auth/login validates credentials and returns a JWT plus user profile.	Public, then authenticated session
Courses	GET /courses lists courses. GET /courses/{id} returns one course. GET /courses/{id}/modules returns modules.	Authenticated users
Modules	GET /modules/{id}/lessons returns lessons for a module, ordered by lesson order.	Authenticated users
Lessons	GET /lessons/{id} returns detailed lesson content and any attached test metadata.	Authenticated users
Tests	GET /tests/lesson/{lessonId} loads a quiz. POST /tests/submit stores a scored result. GET /tests/results returns personal history.	Authenticated users
Progress	GET /progress returns lesson rows. GET /progress/summary returns totals. POST /progress/complete and /progress/time update records.	Authenticated users
Admin	GET /admin/users, /admin/users/{id}, and /admin/test-results expose learner and assessment reporting.	Admin role only
MAD Cloud	POST /mad-cloud/tasks creates and completes a validation task. GET /mad-cloud/tasks/{id} returns an owned task.	Authenticated users
Health	GET /health reports product, database status, and timestamp.	Operators

18. Troubleshooting Guide

Login problems

- * If login fails with valid-looking credentials, confirm the user exists in the target environment and that the password was not changed.
- * If the browser never reaches the dashboard, check that the frontend is pointed to the correct API URL for the environment.
- * If API calls fail only in the browser but work from direct tools, check CORS origins for the frontend URL.
- * If an Admin cannot see Admin Dashboard, verify that the stored user role is exactly Admin and log out/in to refresh local storage.

Learning and progress problems

- * If lessons do not appear, check course-module-lesson seed data and the module id used by the frontend route.
- * If completion does not update, verify the user is authenticated and the progress complete endpoint can write to the database.
- * If time spent is zero, remember that time is updated when a lesson is completed or when the lesson component is destroyed after at least one minute.

- * If test results do not show, confirm the test submission returned successfully and that results are queried for the same user id.

Deployment and rendering problems

- * If the frontend is blank, inspect browser console errors for missing lazy-loaded JavaScript chunks or wrong content types.
- * If a missing chunk returns HTML instead of JavaScript, static hosting is falling through to index.html for a file that should exist.
- * If the API starts but seeding fails, the health endpoint may still respond; check application logs for migration or seed warnings.
- * If local browser tests fail on 127.0.0.1 but pass on localhost, add both origins to development CORS.

19. Glossary

Area	What it does	Who uses it
Course	A complete training program, such as Full-Stack Developer Bootcamp.	Top-level learning unit
Module	A day or topic grouping inside a course.	Course organization
Lesson	The actual learning material, optional code example, and optional quiz link.	Learner activity
Progress	A per-user record of lesson completion and time spent.	Evidence
TestResult	A stored attempt containing score, pass/fail state, and timestamp.	Assessment evidence
MAD Cloud Task	A user-owned validation job with input, status, output, and timestamps.	Task evidence
Admin	A role with access to global user and test reporting.	Permission
JWT	The signed bearer token used to authenticate API requests after login.	Security

20. Known Current Limitations

- * The current course catalog is seeded rather than managed through a full course-authoring UI.
- * Tests support multiple-choice scoring but not question banks, timed attempts, retakes policy, or explanations per answer.
- * Progress is lesson-level and time-minutes based; there is no cohort, assignment, certificate, or manager approval layer yet.
- * MAD Cloud task execution is currently a local completion stub; it does not yet dispatch to a real job queue or external worker network.
- * Admin reporting is useful but still lightweight; exports, filters, cohorts, and trend analytics are natural next steps.
- * The current app supports Student and Admin roles only; larger deployments may need tenant, manager, instructor, assessor, and client roles.
- * The frontend currently depends on correctly deployed lazy-loaded chunks; the deployment process should always verify that every chunk referenced by index.html and route bundles exists.